Zurich University of Applied Sciences

Department of Life Sciences and Facility Management

Institute for Computational Life Sciences

Technical Project Report

Interactive Air Quality Map Dashboard

Communication and Collaboration in Environmental Science
Group Assignment, Spring 2026

Authors:

Lejla Beganovic
Florian Klaver
Claudio Wismer

Supervisor:

Patrick Laube

Submitted on

2026-05-31

Table of Contents

1 Introduction

Air quality is one of the most pressing environmental and public health challenges in Europe. According to the World Health Organization, long-term exposure to particulate matter (PM2.5 and PM10) is responsible for millions of premature deaths globally each year (World Health Organization 2021), and the European Environment Agency (EEA) consistently identifies air pollution as the single largest environmental health risk across the continent. Concentrations of PM10, PM2.5, nitrogen dioxide (NO2), and ozone (O3) vary substantially across geography, seasons, and years — driven by industrial emissions, residential heating, road traffic, and meteorological conditions such as temperature inversions and long-range transport of Saharan dust. Making this complexity legible through interactive spatial and temporal visualisation is an essential tool for both scientific communication and evidence-based policy monitoring.

This project develops an **interactive, map-centred dashboard** built on open data from the **EEA Air Quality e-Reporting (AQeR)** programme (European Environment Agency 2024), the authoritative pan-European monitoring network covering more than 40 countries and thousands of individual monitoring stations. The project was developed as part of the ZHAW module *Communication and Collaboration in Environmental Science* (CCES FS26), with the explicit aim of applying modern scientific communication techniques — interactive visualisation, animated playback, and AI-assisted interpretation — to a real-world environmental dataset. The complete source code is maintained in a public repository at `github.com/[username]/air-quality-dashboard` and the dashboard is launched with a single terminal command (see Section 4).

The central question guiding the design is:

> *How have PM10 and PM2.5 concentrations evolved across Europe between 2013 and 2024, and what does the current (live) situation look like at station level?*

The dashboard answers this through two complementary views: a **Historic tab** for long-term trend exploration with animated playback across more than a decade of daily station measurements, and a **Live tab** for near-real-time monitoring of the most recent seven days at individual station level with hourly resolution.

Extension topic — Animation. The chosen extension topic is *Animation*. The Historic tab implements a full Play/Stop animation engine that steps through daily snapshots of the entire monitoring network, rendering colour-coded station dots on a WebGL map via deck.gl (vis.gl / OpenJS Foundation 2024). A key technical constraint was that the map must not reset its zoom or pan state between frames — solved by pushing data updates through deck.gl's `setProps()` API rather than re-mounting the map element. The result is a fluid temporal narrative of how pollution evolves through seasons and across years, offering qualitatively different insight from any static snapshot.

2 Storytelling Concept and Intended Insights

Effective data visualisation is not only about technical accuracy — it is about guiding an audience toward meaningful understanding through deliberate narrative structure. The design of this dashboard was shaped by three core tensions embedded in the data, each of which motivates a specific design decision.

Tension 1: Then versus now. The split between the Historic and Live tabs creates a temporal contrast that invites users to connect long-term trends with the present situation. A user can observe that Poland has shown a gradual improvement in PM10 over the decade — and then immediately switch to the Live tab to see what Polish stations are reporting today. This past-versus-present structure is a well-established narrative device in environmental communication, emphasising that historical trends are not merely academic but have a direct counterpart in the current world.

Tension 2: East versus West. The spatial embedding of the data on a real map makes immediately visible the persistent disparity in air quality between Eastern and Western Europe. Countries such as Poland, Bulgaria, and Romania show systematically higher PM concentrations, reflecting continued reliance on coal and biomass for residential heating (European Commission 2016). This geographic inequality — which would require multiple paragraphs to convey in prose — is grasped in seconds on the map. The dashboard thereby communicates a message with direct relevance to EU environmental policy: despite measurable continent-wide improvement, geographic equity in air quality remains unresolved.

Tension 3: Urban versus rural. Station dots are visually differentiated by area type — urban (solid circle), suburban (circle with white border ring), and rural (hollow ring). This encoding makes the urban–rural pollution gradient visible at every zoom level without requiring active user filtering. Research consistently shows that urban populations face significantly higher pollutant exposures than rural populations (World Health Organization 2021); the map makes this gradient tangible at the level of individual stations.

The **intended primary audience** is environmental scientists, policy analysts, and informed members of the public with an interest in European air quality trends. The dashboard assumes no prior knowledge of the EEA monitoring network but expects basic familiarity with interactive maps and time series. A user completing a full exploration session should arrive at the following key insights:

1. PM10 and PM2.5 concentrations have declined across most of Europe since 2013, but the pace of improvement is geographically uneven.
2. Winter peaks in particulate concentrations are a structural feature of European air quality, driven primarily by residential heating emissions.
3. Significant country- and city-level disparities persist, with Eastern European stations consistently reporting higher values.

4. Urban monitoring stations record markedly higher pollutant levels than rural stations in the same region.

The **AI-powered peak explanation** feature serves a specific narrative function: it bridges the gap between an observed data pattern and real-world events. When a user notices a spike in the daily average chart, the "Ask AI" button queries the Google Gemini API (Google DeepMind 2024) and returns a short contextual explanation — for example, citing a Saharan dust intrusion or a prolonged temperature-inversion episode. This transforms the dashboard from a passive display tool into an active explanatory environment, supporting deeper scientific understanding without requiring the user to leave the application.

3 Key Dashboard Functionalities

The dashboard is organised into two tabs, each targeting a distinct analytical question. The following overview describes the principal interactive features.

3.1 Historic Tab

Animated map playback. A Play/Stop control steps through daily snapshots of all stations between the user-selected start and end dates. Speed is controlled by a five-position slider corresponding to 0.1 – 1.2 seconds per frame. The current date advances in real time and is displayed above the map. Critically, zoom and pan state are preserved across all animation frames via deck.gl's `setProps()` rather than a full map remount — the core design decision that makes smooth, exploratory animation possible.

EAQI colour coding. Each station dot is coloured according to the official six-tier European Air Quality Index (European Environment Agency 2023), using a colourblind-friendly diverging palette (blue → yellow → red). Missing values are rendered as fully transparent dots rather than omitted, preserving the visual structure of the monitoring network and making data gaps explicit to the user.

Area-type differentiation. Urban, suburban, and rural stations are rendered with distinct visual encodings (solid, border-ring, and hollow circles respectively), making the urban–rural gradient legible at any zoom level without requiring explicit filtering.

Country filter with automatic zoom. A dropdown covering all 29 represented countries restricts the map, charts, and summary statistics to that country's stations, and smoothly zooms the map to the country's bounding box. All charts update accordingly within the same reactive graph.

Daily average time series with interactive date selection. An Altair chart shows the network-wide (or country-filtered) daily mean concentration. Clicking any point on this chart jumps the map to the corresponding date, enabling seamless back-and-forth navigation between chart and map without breaking the animation state.

Year-over-Year comparison chart. Up to five representative years are selected automatically and overlaid on a shared January–December axis, making seasonal structure and inter-annual change directly comparable at a glance.

AI-powered peak explanation. A dedicated button sends the currently displayed date and pollutant to the Google Gemini API and displays a brief contextual explanation of likely causal events, supporting interpretation beyond what the data alone can show.

3.2 Live Tab

Seven-day live station map. EEA Parquet files for the most recent seven days are downloaded concurrently using a 20-thread pool executor, parsed, and rendered as a colour-coded station map. Data is fetched per country and pollutant combination on demand.

Station comparison panel. Clicking two stations on the map opens a side-by-side hourly time-series comparison. EAQI quality bands are rendered as translucent background rectangles, providing immediate visual calibration. A stacked bar chart summarises the proportion of hours in each quality category over the seven-day window.

Pollutant selector. Users can switch between PM10, PM2.5, NO2, and O3. All colours, thresholds, and data queries update dynamically, using pollutant-specific EAQI breakpoints throughout.

4 Setup and Configuration

This document contains the complete annotated source code for the Air Quality Dashboard. The application is split across four Python modules (`app.py`, `hist_tab.py`, `live_tab.py`, and a shared utilities module). The full codebase is available at the repository URL listed on the title page. Run the dashboard from the project root with:

```
shiny run --reload app.py
```

4.1 Required Packages

The following Python packages are required. Run this block once if not already installed.

```
import subprocess, sys

packages = [
    "shiny", "shinywidgets", "duckdb", "pandas",
    "altair", "requests", "pyarrow", "google-genai", "python-dotenv",
]
for pkg in packages:
    subprocess.check_call([sys.executable, "-m", "pip", "install", pkg])
```

4.2 Load Libraries and Define Global Configuration

```
import os, io, gzip
from functools import lru_cache
from datetime import datetime, timedelta, timezone
import datetime as _dt
from concurrent.futures import ThreadPoolExecutor

import requests
import pandas as pd
import altair as alt
import duckdb

from shiny import App, ui, render, reactive, req
from shinywidgets import output_widget, render_altair

#   API endpoints
EEA_API_URL = (
    "https://eeadmz1-downloads-api-appservice.azurewebsites.net/ParquetFile/urls"
)
```

```python
#   Pollutants
POLLUTANTS = ["PM10", "PM2.5", "NO2", "O3"]

#   Historic database
DB_PATH = "eeaopt.db"
HIST_TABLES = {"PM10": "airquality_5", "PM2.5": "airquality_6001"}

print("Setup complete.")
```

4.3 European Air Quality Index (EAQI) Configuration

The dashboard uses the official six-tier **European Air Quality Index** (European Environment Agency 2023) as the colour scheme throughout both tabs. The palette follows a diverging blue–yellow–red progression that is both perceptually ordered and broadly colourblind-safe.

```python
EAQI_THRESHOLDS = {
    "PM2.5": [(5,"Good","#4477AA"),(15,"Fair","#77AADD"),(50,"Moderate","#DDCC77"),
              (90,"Poor","#EE7733"),(140,"Very poor","#CC3311"),
              (float("inf"),"Extremely poor","#882255")],
    "PM10":  [(15,"Good","#4477AA"),(45,"Fair","#77AADD"),(120,"Moderate","#DDCC77"),
              (195,"Poor","#EE7733"),(270,"Very poor","#CC3311"),
              (float("inf"),"Extremely poor","#882255")],
    "NO2":   [(10,"Good","#4477AA"),(25,"Fair","#77AADD"),(60,"Moderate","#DDCC77"),
              (100,"Poor","#EE7733"),(150,"Very poor","#CC3311"),
              (float("inf"),"Extremely poor","#882255")],
    "O3":    [(60,"Good","#4477AA"),(100,"Fair","#77AADD"),(120,"Moderate","#DDCC77"),
              (160,"Poor","#EE7733"),(180,"Very poor","#CC3311"),
              (float("inf"),"Extremely poor","#882255")],
}
EAQI_LABELS  = ["Good","Fair","Moderate","Poor","Very poor","Extremely poor"]
EAQI_COLOURS = {"Good":"#4477AA","Fair":"#77AADD","Moderate":"#DDCC77",
                "Poor":"#EE7733","Very poor":"#CC3311","Extremely poor":"#882255"}

def get_aqi_label(value, pollutant):
    """Return the EAQI category label for a µg/m³ value."""
    if pd.isna(value) or value < 0:
        return None
    for upper, label, _ in EAQI_THRESHOLDS.get(pollutant, EAQI_THRESHOLDS["PM10"]):
        if value <= upper:
            return label
    return "Very poor"
```

5 Datasets

5.1 Historic Database (DuckDB)

The historic data is stored in a local **DuckDB** database (DuckDB Foundation 2024). DuckDB is an in-process analytical database optimised for column-oriented queries — well suited for aggregating millions of daily station measurements without the overhead of a separate server process. The database was populated from EEA AQeR bulk downloads and contains two tables:

- `airquality_5` — PM10 daily measurements, 29 European countries, 2013–2024
- `airquality_6001` — PM2.5 daily measurements, 29 European countries, 2013–2024

Each row contains the columns `Date`, `Value` (in µg/m³), `Country` (ISO 3166-1 alpha-2 code), `Latitude`, and `Longitude`. The combined dataset spans over 40 million measurement records, making efficient columnar storage and query caching essential for interactive response times.

All queries against the historic database are wrapped with Python's `@lru_cache` decorator using the date string and table name as cache keys, so that repeating an animation frame or moving back to a previously visited date requires no additional database I/O. This design pattern is detailed further in the Historic Tab section.

```
_db_conn = None

def get_db():
    """Return a cached read-only DuckDB connection."""
    global _db_conn
    if _db_conn is None:
        _db_conn = duckdb.connect(DB_PATH, read_only=True)
    return _db_conn
```

5.1.1 Inspecting the Database Schema

```
conn = get_db()

schema = conn.execute("DESCRIBE airquality_5").fetch_df()
print(schema.to_string(index=False))

summary = conn.execute("""
    SELECT COUNT(*) AS n_rows,
           MIN(DATE("Date")) AS first_date,
           MAX(DATE("Date")) AS last_date,
           COUNT(DISTINCT "Country") AS n_countries
    FROM airquality_5
    WHERE "Value" IS NOT NULL
```

```
""").fetch_df()
print(summary)
```

5.1.2 Station Metadata

A supplementary CSV file (`station_metadata_clean.csv`) links station sampling-point IDs to human-readable names, geographic coordinates, and **area type** (urban, suburban, rural). This classification is the basis for the three-symbol visual differentiation on the map and was prepared in a dedicated preprocessing notebook (`metadata_prep.ipynb`). The area-type field follows the EEA's station classification vocabulary. Approximately 5% of live station IDs could not be matched to the metadata CSV; these stations are excluded from the map but are retained in time-series charts where coordinate information is not required.

```
@lru_cache(maxsize=1)
def get_station_coords():
    """Load and cache station metadata. Falls back to EEA API if CSV absent."""
    try:
        df = pd.read_csv("station_metadata_clean.csv", low_memory=False)
        # Normalise column names (strip BOM, lowercase, replace separators)
        df.columns = (df.columns.astype(str)
                      .str.replace("\ufeff", "", regex=False).str.strip()
                      .str.lower()
                      .str.replace(r"[\s\-\.\/]+", "_", regex=True))
        print(f"Loaded {len(df)} station records.")
        return df
    except FileNotFoundError:
        print("CSV not found - fetching from EEA API …")
        return pd.DataFrame(columns=["key","lat","lon","station_name","area_type"])
```

5.2 Live Data (EEA Parquet API)

The Live tab fetches the most recent seven days of hourly measurements directly from the **EEA AQeR download API** (European Environment Agency 2024). A POST request specifies the country code, pollutant, and a rolling seven-day time window ending at the current UTC time. The API returns a list of Parquet file URLs — one per station — which are then downloaded concurrently using a `ThreadPoolExecutor` with 20 worker threads to minimise total loading time. Parquet was chosen as the transfer format because it provides efficient columnar compression and allows column-selective reading without deserialising the full payload.

```
def get_station_urls(country_code, pollutant):
    """Fetch Parquet file URLs for a given country and pollutant."""
```

```
    end_dt   = datetime.now(timezone.utc)
    start_dt = end_dt - timedelta(days=7)
    payload  = {
        "countries": [country_code], "cities": [], "pollutants": [pollutant],
        "dataset": 1,
        "dateTimeStart": start_dt.strftime("%Y-%m-%dT%H:%M:%S.000Z"),
        "dateTimeEnd":   end_dt.strftime("%Y-%m-%dT%H:%M:%S.000Z"),
        "compress": False,
    }
    resp = requests.post(EEA_API_URL, json=payload)
    resp.raise_for_status()
    return [l.strip() for l in resp.text.strip().split("\n")
            if l.strip() and "ParquetFileUrl" not in l]


@lru_cache(maxsize=64)
def get_all_station_data_cached(country_code, pollutant):
    """Fetch and cache all station data for a country/pollutant pair."""
    urls   = get_station_urls(country_code, pollutant)
    coords = get_station_coords()
    cutoff = (datetime.now(timezone.utc) - timedelta(days=7)).replace(tzinfo=None)

    def fetch_one(url):
        try:
            df = pd.read_parquet(url)
            if df.empty: return None
            # Parse nanosecond integer or ISO-string timestamps
            if pd.api.types.is_numeric_dtype(df["Start"]):
                df["Start"] = pd.to_datetime(df["Start"], unit="ns", errors="coerce")
            else:
                df["Start"] = pd.to_datetime(df["Start"], errors="coerce")
            df["Start"] = df["Start"].dt.tz_localize(None)
            df["Value"] = pd.to_numeric(df["Value"], errors="coerce")
            df = df[df["Value"] >= 0].dropna(subset=["Start","Value"])
            df = df[df["Start"] >= cutoff].reset_index(drop=True)
            if df.empty: return None
            raw_id     = str(df["Samplingpoint"].iloc[0]) if "Samplingpoint" in df.columns
 ↪  else url
            station_id = raw_id.split("/")[-1] if "/" in raw_id else raw_id
            ts = df[["Start","Value"]].copy()
            ts["station_id"] = station_id
            return {"meta": {"station_id": station_id, "Value": float(df["Value"].mean())},
                    "ts": ts}
        except Exception:
            return None

    with ThreadPoolExecutor(max_workers=20) as ex:
        results = [r for r in ex.map(fetch_one, urls) if r is not None]
    if not results:
```

```
        return pd.DataFrame(), pd.DataFrame()

    df_meta = pd.DataFrame([r["meta"] for r in results])
    df_all  = pd.concat([r["ts"] for r in results], ignore_index=True)
    # Merge coordinates and area type from metadata CSV
    if not coords.empty:
        df_meta["key"] = df_meta["station_id"].str.strip().str.lower()
        df_meta = df_meta.merge(
            coords[["key","lat","lon","station_name","area_type"]],
            on="key", how="left").drop(columns=["key"])
    df_meta["station_name"] = df_meta.get("station_name",
                              df_meta["station_id"]).fillna(df_meta["station_id"])
    if "area_type" not in df_meta.columns:
        df_meta["area_type"] = "unknown"
    df_meta["area_type"] = df_meta["area_type"].fillna("unknown")
    name_map = df_meta.set_index("station_id")["station_name"].to_dict()
    df_all["station_name"] = df_all["station_id"].map(name_map).fillna(
        df_all["station_id"])
    return df_meta, df_all
```

5.3 Data Quality

The raw EEA data contains a number of quality issues encountered during development. The table below summarises the most significant issues and the handling strategy applied in both the preprocessing pipeline and the live fetch functions.

Table 1: Data quality issues and handling strategies

Issue	Handling strategy
Missing daily station values	Rendered with zero opacity, not omitted
Negative values (instrument errors)	Filtered: `Value >= 0`
Mixed timestamp formats (ns int / ISO string)	Unified parser, timezone stripped
Unmatched station coordinates (~5%)	Dropped from map, retained in time series
Duplicate entries per station per day	First occurrence kept after deduplication

6 Historic Tab

6.1 Query Functions

All historic queries run against the local DuckDB database. The query architecture is built around two principles: (1) expensive full-table scans are performed once and cached per table name, and (2) per-date queries are cached using the ISO date string and table name as composite keys. This means that when the animation replays a date it has previously rendered, the result is returned from the Python `lru_cache` in microseconds rather than re-executing a SQL query.

```
@lru_cache(maxsize=4)
def hist_get_all_stations(table_name):
    """
    Load ALL stations for a pollutant table once.
    Avoids repeated DISTINCT scans on every country change.
    Country filtering is done in Python after this single DB call.
    """
    conn = get_db()
    return conn.execute(f"""
        SELECT DISTINCT "Latitude" AS lat, "Longitude" AS lon, "Country"
        FROM {table_name}
        WHERE "Latitude" IS NOT NULL AND "Longitude" IS NOT NULL
    """).fetch_df()


def hist_get_master_stations(table_name, start, end, country="ALL"):
    df = hist_get_all_stations(table_name)
    if country != "ALL":
        df = df[df["Country"] == country]
    return df[["lat","lon"]].reset_index(drop=True)


@lru_cache(maxsize=512)
def hist_get_map_data_cached(date_str, table_name):
    """Fetch all station values for a single date (cached per date)."""
    conn = get_db()
    return conn.execute(f"""
        SELECT "Value", "Latitude" AS lat, "Longitude" AS lon, "Country"
        FROM {table_name}
        WHERE DATE("Date") = '{date_str}'
          AND "Latitude" IS NOT NULL AND "Longitude" IS NOT NULL
    """).fetch_df()


def hist_get_map_data(date_str, table_name, master_df=None, country="ALL"):
    """Retrieve and EAQI-colour-code station values for a specific date."""
    df = hist_get_map_data_cached(date_str, table_name)
```

```
    if country != "ALL":
        df = df[df["Country"] == country]
    if master_df is not None and not master_df.empty:
        df = df.drop_duplicates(subset=["lat","lon"])
        df = pd.merge(master_df, df, on=["lat","lon"], how="left")
    pollutant = "PM2.5" if "6001" in table_name else "PM10"
    if not df.empty and "Value" in df.columns:
        thresholds = EAQI_THRESHOLDS.get(pollutant)
        def _color(v):
            if pd.isna(v) or v <= 0: return (128, 128, 128, 0)
            for upper, _, hx in thresholds:
                if v <= upper:
                    return (int(hx[1:3],16), int(hx[3:5],16),
                            int(hx[5:7],16), 210)
            return (136, 34, 85, 210)
        cols = df["Value"].apply(_color)
        df["color_r"]   = cols.apply(lambda c: c[0]).astype(int)
        df["color_g"]   = cols.apply(lambda c: c[1]).astype(int)
        df["color_b"]   = cols.apply(lambda c: c[2]).astype(int)
        df["color_a"]   = cols.apply(lambda c: c[3]).astype(int)
        df["value_str"] = df["Value"].apply(lambda x: f"{x:.2f}" if pd.notna(x) else "N/A")
        df["aqi_label"] = df["Value"].apply(lambda v: get_aqi_label(v, pollutant) or "No
↪   data")
    return df


@lru_cache(maxsize=32)
def hist_get_daily_averages(table_name, start, end):
    """Daily mean value across all stations for the time-series chart."""
    conn = get_db()
    df = conn.execute(f"""
        SELECT DATE("Date") AS "Date", AVG("Value") AS AvgValue
        FROM {table_name}
        WHERE "Value" IS NOT NULL
          AND DATE("Date") BETWEEN '{start}' AND '{end}'
        GROUP BY DATE("Date") ORDER BY "Date"
    """).fetch_df()
    df["Date"] = pd.to_datetime(df["Date"])
    return df


@lru_cache(maxsize=16)
def hist_get_yoy_data(table_name, start, end):
    """
    Monthly averages per calendar year for the Year-over-Year chart.
    Years are pinned to 2000 so Altair overlays them on a shared
    January-December x-axis.
    """
    conn = get_db()
```

```
    df = conn.execute(f"""
        SELECT YEAR("Date") AS Year, MONTH("Date") AS Month,
               AVG("Value") AS AvgValue
        FROM {table_name}
        WHERE "Value" IS NOT NULL
          AND DATE("Date") BETWEEN '{start}' AND '{end}'
        GROUP BY Year, Month ORDER BY Year, Month
    """).fetch_df()
    df["YearStr"]   = df["Year"].astype(str)
    df["MonthDate"] = pd.to_datetime(
        "2000-" + df["Month"].astype(str).str.zfill(2) + "-15")
    return df
```

6.2 Animation Engine

The animation engine is the technical centrepiece of the extension topic implementation. The key design decision is to use `reactive.invalidate_later()` rather than a blocking `while` loop: this keeps Shiny's event loop fully responsive during playback, allowing the user to change the speed, stop the animation, or interact with any other UI element at any time. Each invocation of `_hist_advance()` schedules its own next execution, effectively creating a non-blocking timer that fires once per frame at the user-selected interval.

```
# Speed mapping: slider position 1-5 → seconds per frame
_SPEED_MAP = {1: 1.2, 2: 0.8, 3: 0.5, 4: 0.25, 5: 0.1}

# Excerpt from hist_server() in hist_tab.py:
def hist_server_animation_excerpt(input, output, session):

    hist_playing   = reactive.Value(False)
    hist_anim_date = reactive.Value(None)

    @reactive.effect
    @reactive.event(input.hist_play)
    def _hist_play():
        hist_playing.set(True)
        hist_anim_date.set(input.hist_date())

    @reactive.effect
    @reactive.event(input.hist_stop)
    def _hist_stop():
        hist_playing.set(False)

    @reactive.effect
    def _hist_advance():
        if not hist_playing():
```

```
            return
        speed = _SPEED_MAP.get(input.hist_speed(), 0.5)
        reactive.invalidate_later(speed)          # re-fires after `speed` seconds
        with reactive.isolate():
            cur = hist_anim_date() or input.hist_date()
        nxt = cur + _dt.timedelta(days=1)
        if nxt > input.hist_end():
            hist_playing.set(False)
        else:
            hist_anim_date.set(nxt)               # triggers all downstream reactives
```

6.3 Map Rendering: deck.gl + MapLibre GL

The map is a static `<div id="deck-hist-map">` mounted once at page load. **deck.gl** (vis.gl / OpenJS Foundation 2024) renders three `ScatterplotLayer` instances — one each for urban, suburban, and rural stations — on a **MapLibre GL** (MapLibre Contributors 2024) base map. Data updates arrive via Shiny's custom WebSocket message channel and are applied with `setProps()`, meaning no re-mount occurs across animation frames or country changes and the user's zoom and pan state is fully preserved throughout.

```
_HIST_JS_COLS = ["lon","lat","color_r","color_g","color_b","color_a",
                 "value_str","station_name","aqi_label","area_type"]

def build_hist_map_payload(df):
    """Split styled DataFrame into urban/suburban/rural for deck.gl."""
    payload = {"urban":[],"suburban":[],"rural":[],"stamp":0}
    if df.empty: return payload
    if "area_type" in df.columns:
        norm     = df["area_type"].str.lower().fillna("unknown")
        urban    = df[norm.isin(["urban","unknown"])]
        suburban = df[norm == "suburban"]
        rural    = df[norm.isin(["rural","rural-nearcity","rural_nearcity"])]
    else:
        urban, suburban, rural = df, df.iloc[0:0], df.iloc[0:0]
    def to_records(sub):
        if sub.empty: return []
        cols = [c for c in _HIST_JS_COLS if c in sub.columns]
        return sub[cols].where(sub[cols].notna(), None).to_dict("records")
    payload["urban"]    = to_records(urban)
    payload["suburban"] = to_records(suburban)
    payload["rural"]    = to_records(rural)
    payload["stamp"]    = datetime.utcnow().timestamp()
    return payload
```

6.4 Chart Builders

```
def build_hist_avg_chart(df_avg, current_date):
    """
    Daily-average time-series chart.
    The red dot is driven by a Vega parameter updated via a lightweight
    signal message - no chart re-embed on every animation frame,
    so zoom and pan state are preserved.
    A click-selection layer lets users jump to any date by clicking.
    """
    if df_avg.empty:
        return (alt.Chart(pd.DataFrame({"Date":[],"AvgValue":[]}))
                .mark_line().properties(height=300).to_dict())
    df = df_avg.copy()
    df["DateStr"]  = df["Date"].dt.strftime("%Y-%m-%d")
    date_param     = alt.param(name="histCurDate", value=str(current_date))
    click_sel      = alt.selection_point(
        name="date_click", on="click", nearest=True, fields=["DateStr"])
    click_target   = (alt.Chart(df).mark_point(opacity=0.001, size=300)
                      .encode(x="Date:T", y="AvgValue:Q")
                      .add_params(click_sel))
    base = (alt.Chart(df).mark_line(color="gray", strokeWidth=2)
            .encode(x=alt.X("Date:T", title="Date"),
                    y=alt.Y("AvgValue:Q", title="Daily Average (µg/m³)"))
            .properties(height=300, title="Daily Average Values Over Time")
            .interactive())
    dot = (alt.Chart(df).mark_circle(color="red", size=100, opacity=1)
           .encode(x="Date:T", y="AvgValue:Q")
           .transform_filter("datum.DateStr === histCurDate"))
    spec = (alt.layer(base, dot, click_target)
            .add_params(date_param).resolve_scale(y="shared")
            .properties(width="container").to_dict())
    spec["autosize"] = {"type":"fit-x","contains":"padding"}
    return spec


def build_yoy_chart(df_yoy, pollutant):
    """
    Year-over-Year seasonal comparison chart.
    Up to five representative years are selected and overlaid on a
    shared January-December x-axis.
    """
    if df_yoy.empty:
        return alt.Chart(pd.DataFrame()).mark_line().properties(height=300)
    years = sorted(df_yoy["Year"].unique())
    if len(years) > 5:
        step  = max(1, len(years) // 4)
        picked = years[::step]
        if years[-1] not in picked:
```

```
            picked = list(picked) + [years[-1]]
        df_yoy = df_yoy[df_yoy["Year"].isin(picked)]
    return (
        alt.Chart(df_yoy).mark_line(point=True, strokeWidth=2)
        .encode(
            x=alt.X("MonthDate:T", title="Month",
                    axis=alt.Axis(format="%b", tickCount="month", labelAngle=0)),
            y=alt.Y("AvgValue:Q", title=f"{pollutant} Monthly Avg (µg/m³)"),
            color=alt.Color("YearStr:N", title="Year"),
            tooltip=[alt.Tooltip("YearStr:N", title="Year"),
                     alt.Tooltip("MonthDate:T", format="%B", title="Month"),
                     alt.Tooltip("AvgValue:Q", format=".1f", title="µg/m³")],
        )
        .properties(height=300, title="Year-over-Year Comparison (monthly averages)")
        .interactive().properties(width="container")
    )
```

7 Live Tab

7.1 AI-Powered Peak Explanation

The Historic tab includes an "Ask AI" feature that sends the currently selected date and pollutant to the **Google Gemini API** (Google DeepMind 2024). The model (`gemini-2.5-flash`) returns a 1–2 sentence contextual explanation of what event likely caused the observed pollution peak. The function is deliberately kept narrow in scope: it provides environmental context (e.g. meteorological events, dust transport episodes, seasonal heating onset) rather than general air quality information, keeping responses concise and directly actionable for the user.

```python
from google import genai

def ask_llm_about_peak(date_str, metric):
    """Query Google Gemini for a contextual peak explanation."""
    if not os.environ.get("GEMINI_API_KEY"):
        return "Error: GEMINI_API_KEY not set."
    try:
        client   = genai.Client()
        prompt   = (
            f"Explain briefly in only 1-2 sentences what event could cause "
            f"there to be an air quality ({metric}) peak on {date_str} "
            f"in Europe. Keep your response short and concise."
        )
        response = client.models.generate_content(
            model="gemini-2.5-flash", contents=prompt)
        return response.text
    except Exception as e:
        return f"Error communicating with Gemini API: {e}"
```

7.2 Live Station Comparison

Clicking two stations on the live map opens a comparison panel containing an hourly time series overlaid with EAQI band backgrounds and a stacked bar chart showing the proportion of hours in each quality category over the seven-day window. The band backgrounds are constructed programmatically from the pollutant-specific EAQI thresholds, so the chart is correct for all four supported pollutants without any hard-coded values.

```python
def build_comparison_chart(df_sel, pollutant):
    """Hourly time series with coloured EAQI band backgrounds."""
    thresholds = EAQI_THRESHOLDS.get(pollutant, EAQI_THRESHOLDS["PM10"])
    y_max = max(float(df_sel["Value"].max()) * 1.15, thresholds[1][0] + 1)
    band_data, prev = [], 0
```

```
    for upper, label, colour in thresholds:
        y2 = min(upper, y_max) if upper != float("inf") else y_max
        if prev >= y_max: break
        band_data.append({"y1":float(prev),"y2":float(y2),
                          "label":label,"colour":colour})
        prev = upper if upper != float("inf") else y_max
    aqi_scale = alt.Scale(domain=[r["label"] for r in band_data],
                          range=[r["colour"] for r in band_data])
    bands = (alt.Chart(pd.DataFrame(band_data)).mark_rect(opacity=0.15)
             .encode(y=alt.Y("y1:Q", scale=alt.Scale(domain=[0, y_max])),
                     y2="y2:Q",
                     color=alt.Color("label:N", scale=aqi_scale, legend=None)))
    line = (alt.Chart(df_sel).mark_line(point=alt.OverlayMarkDef(size=30))
            .encode(x=alt.X("Start:T", title="Date & Time"),
                    y=alt.Y("Value:Q", title=f"{pollutant} (µg/m³)",
                            scale=alt.Scale(domain=[0, y_max])),
                    color=alt.Color("station_name:N", title="Station"),
                    tooltip=[alt.Tooltip("Start:T", format="%d %b, %H:%M"),
                             alt.Tooltip("Value:Q", format=".2f", title="µg/m³"),
                             alt.Tooltip("station_name:N", title="Station")])
            .properties(height=300, title="Hourly Readings with AQI Zones")
            .interactive())
    return alt.layer(bands, line).resolve_scale(color="independent")
```

7.3 Application Entry Point

The four modules are assembled in `app.py`, which defines the top-level UI and server functions and passes them to Shiny's `App` constructor. The shared CSS supports both dark and light mode via `@media (prefers-color-scheme: light)`. CDN-hosted JavaScript libraries (MapLibre GL, deck.gl) are loaded in the HTML `<head>` to avoid bundling them with the Python package.

```
# app.py - entry point
from shiny import App, ui
from live_tab import live_ui, live_server
from hist_tab import hist_ui, hist_server

app_ui = ui.page_fluid(
    ui.head_content(
        ui.tags.link(rel="stylesheet",
            href="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.css"),
        ui.tags.script(
            src="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.js"),
        ui.tags.script(
            src="https://unpkg.com/deck.gl@9.0.33/dist.min.js"),
    ),
    ui.h1("Air Quality Map Dashboard"),
```

```
    ui.navset_tab(
        live_ui(),   # Live Data tab
        hist_ui(),   # Historic Data tab
        selected="Live Data",
    ),
)

def server(input, output, session):
    live_server(input, output, session)
    hist_server(input, output, session)

app = App(app_ui, server)
```

8 Discussion and Conclusion

8.1 Key Findings

The dashboard confirms and makes visually tractable several patterns well established in the European air quality literature. The animated spatial view immediately reveals that **Eastern European countries** — particularly Poland, Bulgaria, and Romania — show systematically higher PM10 concentrations than Western Europe (European Commission 2016), consistent with continued reliance on coal and biomass for residential heating. This East–West disparity persists even at the end of the 2013–2024 observation window, suggesting that the long-term downward trend has not yet closed the geographic gap.

The Year-over-Year comparison chart provides a clear summary of the **long-term improvement trend** across Europe. Most years show a stepwise reduction in peak winter values relative to the same months one or two years earlier, consistent with the effects of EU emission reduction directives on the residential and industrial sectors. The seasonal structure — pronounced winter peaks in December–February, substantially lower summer values — is consistent across all countries and driven primarily by residential heating combined with temperature-inversion meteorology that traps emissions near the surface.

The three-symbol area-type differentiation confirms that **urban monitoring stations report consistently higher values** than suburban or rural sites within the same country and region. This finding has direct implications for population exposure assessment: European residents living in urban centres face meaningfully higher long-term pollutant burdens than the country-level statistics alone would suggest.

8.2 Limitations

The following limitations should be considered when interpreting the dashboard outputs.

- **Daily averaging**: Intra-day peaks driven by traffic rush hours or episodic industrial releases are averaged out at the daily granularity stored in the historic database.
- **Live API coverage**: Not all countries report all four pollutants via the EEA live API; some country–pollutant combinations return zero station results.
- **Coordinate matching**: Approximately 5% of live stations cannot be placed on the map due to unresolved sampling-point ID mismatches between the Parquet metadata and the station CSV.
- **AI explanations**: Gemini-generated peak explanations are contextually plausible but are not verified against official EEA event logs or peer-reviewed post-hoc analyses. They should be treated as indicative interpretations rather than authoritative attribution.

8.3 Conclusion

The dashboard successfully communicates twelve years of European air quality data through animated spatial visualisation, seasonal comparison charts, live station monitoring, and AI-assisted interpretation. The animation extension delivers a compelling narrative tool for exploring how pollution evolves across Europe with the seasons and responds to decade-scale policy changes — an insight that no static chart can convey with equivalent immediacy. Future extensions could incorporate weather reanalysis overlays (ERA5 wind fields and boundary-layer height), country-level regression trend lines, a mobile-optimised layout, and automated alerting when live station values cross EAQI thresholds.

9 Declaration of Individual Contributions

Each team member confirms that the following table accurately reflects their contributions to the project.

Table 2: Individual contributions by team member

Task	Lejla	Florian	Claudio
Project architecture & Shiny setup		✓	
Historic tab — map (deck.gl)	✓	✓	
Historic tab — animation engine	✓	✓	✓
Historic tab — charts (Altair)	✓	✓	✓
Historic tab — AI integration			✓
Historic tab — country filter & auto-zoom	✓	✓	✓
Live tab — EEA API integration		✓	✓
Live tab — map & station comparison	✓	✓	✓
Station metadata preprocessing		✓	
Performance optimisation (lru_cache)	✓	✓	✓
Bug testing & quality assurance	✓	✓	✓
Report writing	✓	✓	✓
Screencast / video	✓	✓	✓

10 Declaration of AI Use

Parts of this implementation were developed with assistance from **Claude** (Anthropic, `claude.ai`). Specific areas of AI-assisted development include:

- Conversion of the initial Streamlit prototype to Python Shiny
- Implementation of the deck.gl / MapLibre GL map integration
- Design of the non-blocking animation engine using `reactive.invalidate_later()`
- Development of the Vega signal-based chart dot update mechanism (preserving zoom state across animation frames)
- Performance optimisation through `lru_cache` strategies for DuckDB queries
- Debugging of WebSocket message handling between Python and JavaScript

All AI-generated code was reviewed, tested, and adapted by the team before integration into the codebase. All scientific and design decisions — including EAQI threshold definitions, chart type selection, storytelling structure, and result interpretation — were made independently by the team.

The **"Ask AI about peaks"** dashboard feature uses the Google Gemini API (`gemini-2.5-flash`) at runtime to generate contextual explanations of pollution peak events. This is an intentional product feature, not a development tool, and is described as such in Section 3 (Key Dashboard Functionalities) and Section 7 (Live Tab).

No AI tools were used for data collection, data preprocessing, statistical analysis, or interpretation of the scientific findings presented in this report.

References

DuckDB Foundation. 2024. "DuckDB: An in-Process SQL OLAP Database Management System." https://duckdb.org.

European Commission. 2016. "Directive (EU) 2016/2284 on the Reduction of National Emissions of Certain Atmospheric Pollutants (NEC Directive)." Official Journal of the European Union.

European Environment Agency. 2023. "European Air Quality Index — Methodology." https://www.eea.europa.eu/themes/air/air-quality-index.

———. 2024. "Air Quality e-Reporting (AQeR) — Parquet Download API." https://eeadmz1-downloads-api-appservice.azurewebsites.net.

Google DeepMind. 2024. "Gemini 2.5 Flash — Language Model API." https://ai.google.dev.

MapLibre Contributors. 2024. "MapLibre GL JS — Open-Source Map Rendering." https://maplibre.org.

vis.gl / OpenJS Foundation. 2024. "Deck.gl — WebGL-Powered Large-Scale Data Visualization." https://deck.gl.

World Health Organization. 2021. "WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide." *WHO Report*.